



UNIVERSIDAD DE GRANADA

Facultad de Ciencias

GRADO EN FÍSICA

TRABAJO FIN DE GRADO

Clasificación de jets de partículas subatómicas mediante Machine Learning con el conjunto de datos JetClass

Previamente titulado: Detección de obstáculos en tiempo
real mediante técnicas de Machine Learning

Presentado por:
D^a. Ana Fuentes Rodríguez

Curso Académico 2023/2024

Resumen

Este trabajo tiene como objetivo final el análisis del rendimiento y la eficiencia de un modelo de clasificación de jets de partículas subatómicas a partir del conjunto de datos *JetClass*, aplicando un algoritmo de aprendizaje automatizado. Según las características que cumplan los jets (masa, número de partículas de cada jet, etc.) y las partículas que los forman (hadrones con carga o sin ella, electrones, fotones, etc.), han sido clasificados mediante una red neuronal densamente conectada en diversas categorías (el decaimiento de un bosón de Higgs en dos quarks bottom, o en dos quarks charm, etc.). De esta forma, dependiendo del grado de error que cometa la red clasificando los jets se ha podido construir el modelo más eficiente y preciso posible.

Abstract

The final objective of this work is the analysis of the performance and efficiency of a subatomic particle jet classification model based on the *JetClass* data set, applying a machine learning algorithm. According to the characteristics of the jets (mass, number of particles in each jet, etc.) and the particles that form them (hadrons with or without charge, electrons, photons, etc.), they have been classified using a densely packed neural network. connected in various categories (the decay of a Higgs boson into two bottom quarks, or into two charm quarks, etc.). In this way, depending on the degree of error that the network makes when classifying the jets, it has been possible to build the most efficient and accurate model possible.

Índice

1	Introducción	4
2	Objetivos	8
3	Contribuciones	8
4	Materiales y metodología	9
4.1	Materiales	9
4.2	Metodología	9
4.2.1	Paso previo a la realización del código: Análisis del contenido de los archivos <code>.root</code>	9
4.2.2	Implementación y entrenamiento del modelo	10
4.2.3	Evaluación del modelo	15
5	Resultados y discusión	16
5.1	Entrenamiento del modelo	16
5.1.1	Variación del número de neuronas	16
5.1.2	Variación del número de capas ocultas	17
5.1.3	Variación del tipo de función de activación	18
5.2	Evaluación del modelo final	19
5.3	Comparación con los resultados aportados por los creadores de <i>Jet-Class</i>	20
6	Conclusiones	21
7	Agradecimientos	21
	Referencias	22

1 Introducción

La introducción del aprendizaje automatizado (machine learning) y el uso de redes neuronales de clasificación multidimensionales en la Física de Partículas ha resultado ser un acierto crucial en el avance del estudio de este campo.

Tradicionalmente, este etiquetado de partículas o *jet tagging* se ha hecho manualmente mediante la cromodinámica cuántica (*Quantum ChromoDynamics*, QCD), la cual es una teoría que subyace a la interacción fuerte entre gluones (mediadores de la interacción fuerte) y quarks, dictando así la estructura de núcleos y hadrones [1]. Este procedimiento puede llegar a ser un tanto engorroso, debido a que, al hacerse manualmente, se necesita mucho tiempo y se pueden cometer errores humanos.

Así pues, el empleo del machine learning mejora considerablemente el rendimiento y la eficiencia de la clasificación mediante jets (pulverización colimada de partículas).

La clasificación de jets de partículas mediante machine learning ya ha sido estudiado, sin embargo, no existía un grupo de datos público lo suficientemente amplio como para poder profundizar en el estudio de la clasificación de partículas mediante aprendizaje automatizado.

Para este trabajo, no obstante, se utiliza un conjunto de datos a gran escala llamado *JetClass* [2] con el que se pretende conseguir un avance significativo en la optimización de redes neuronales para el etiquetado de jets.

Este conjunto de datos contiene 100 millones de jets para entrenar la red, 20 millones para probarla y 5 millones para su validación, etiquetando todos estos jets en 10 categorías. Debido a que todos estos jets se producen tras colisiones de alta energía, las clases en los que se clasifican son las siguientes:

1. QCD: son jets generados por quarks y gluones.
2. hbb: jets generados del decaimiento de un bosón de Higgs en dos quarks tipo bottom.
3. hcc: jets producidos tras el decaimiento de un bosón de Higgs en dos quarks tipo charm.
4. hgg: jets generados del decaimiento de nuevo de un bosón de Higgs en dos gluones (partículas mediadoras de la interacción fuerte).
5. H4q: jets que provienen del decaimiento de un bosón de Higgs en cuatro quarks.
6. Hqql: jets que provienen del decaimiento de un bosón de Higgs en dos quarks y un leptón.
7. Zqq: jets que provienen del decaimiento de un bosón Z (mediador de la interacción débil) en dos quarks.

8. Wqq: jets generados del decaimiento de un bosón W (mediador de la interacción débil) en dos quarks.
9. Tbqq: jets que provienen del decaimiento del quark top en un quark bottom y otros dos quarks.
10. Tbl: jets generados del decaimiento de un quark top en un quark bottom y un leptón.

Por otro lado, en este conjunto de datos también se encuentran diversas características de cada uno de los jets y las partículas que los forman. Estas características son:

1. Características de los jets: estas aportan información sobre la descomposición de los jets, la masa después de la colisión, el número de partículas que lo constituyen, la energía, el momento transversal, el ángulo ϕ y la pseudo-rapidez η (ángulo entre una partícula y el eje del haz en un acelerador) del jet.
2. Características de las partículas: estas dan información sobre si es un electrón, un muón, un fotón, un hadrón cargado o neutro, la carga y la energía de la partícula, el valor y el error de la distancia longitudinal z , el valor y el error de la distancia transversal, la diferencia en el ángulo ϕ y η entre la partícula y el jet y las componentes x , y y z de la partícula.
3. Características auxiliares: por otro lado, estas dan información sobre si la partícula está asociada a una partícula simulada o no y el tipo, momento transversal y ángulos ϕ y η de la partícula generada.

Una red neuronal se define como un sistema que establece una relación entre los datos que se le proporcionan a la entrada y con los que se obtienen a la salida, asemejándose este comportamiento al de un cerebro. La diferencia principal entre el aprendizaje automatizado de la programación tradicional es que las redes neuronales no utilizan algoritmos secuenciales. Estas redes neuronales son capaces de procesar la información en paralelo, pudiendo aprender y generalizar situaciones no incluidas en su entrenamiento.

En el cerebro humano, la información se guarda en las zonas de contacto entre las neuronas (la sinapsis), generando así una amplia red de comunicación con la que se llevan a cabo procesos altamente eficaces. De esta manera, Walter Pitts y Warren McCulloch crearon en 1943 el primer modelo de red neuronal artificial, capaz de aprender tácticas y soluciones, tomando ejemplos de patrones típicos de comportamiento [3].

Así pues, se puede definir una red neuronal como un conjunto de neuronas interconectadas entre sí y organizadas en tres capas; el *input* o capa de entrada, por donde se introducen los datos, las *layers* o capas ocultas por donde pasan estos datos y el *output* o capa de salida por donde finalmente salen. En la Figura 1 se muestra un esquema para comprender mejor este sistema.

Las redes neuronales presentan los siguientes elementos básicos:

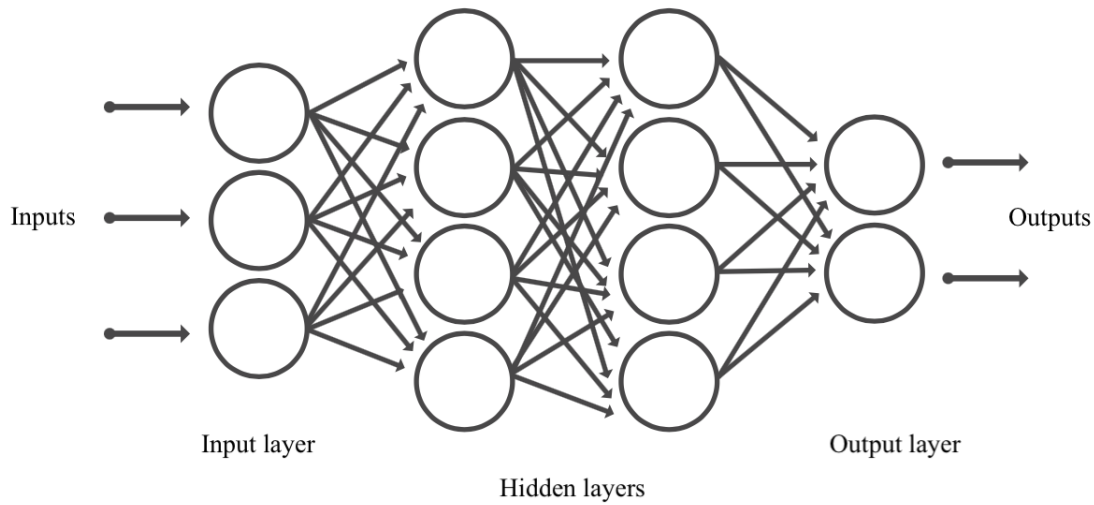


Figura 1: Esquema básico de una red neuronal [4].

- Capas neuronales: como ya se ha explicado, las redes neuronales presentan una capa de entrada (*input layer*), una o varias ocultas (*hidden layers*) y una capa de salida (*output layer*).
- Pesos sinápticos: son características, valores numéricos, que toman las neuronas y que van variando su valor conforme la red se va entrenando.
- Regla de programación: esto hace referencia a cómo está estructurada la red (capas, neuronas, tipos de red, etc), la inicialización de los pesos o el tipo de función de activación que se utiliza en cada capa.
- Función de activación [5]: función que filtra los datos, generando valores que serán los dados por la neurona. Esta función de activación depende del propósito que tenga la red neuronal. Los diferentes tipos de funciones de activación son los siguientes:

1. Función Sigmoide: esta función transforma los valores entrantes a una escala (0,1). Los datos altos tienden asintóticamente a 1 y los muy bajos a 0.

$$f(x) = \frac{1}{1 + e^{-x}} \quad (1.1)$$

Esta función tiene un gran rendimiento en la capa de salida pero tiene una lenta convergencia y no está centrada en 0, estando acotada entre 0 y 1.

2. Función tangente hiperbólica: esta función transforma los valores entrantes a una escala (-1,1), donde los valores altos tienden asintóticamente a 1 y los valores muy bajos a -1.

$$f(x) = \frac{2}{1 + e^{-2x}} - 1 \quad (1.2)$$

Esta función es similar a la anterior, salvo porque está acotada entre -1 y 1 y, por tanto, centrada en 0. Es útil en redes neuronales recurrentes, donde hay que decidir entre una opción o la contraria.

3. Función ReLU (*Rectified Lineal Unit*): esta función transforma los valores entrantes anulando los valores negativos y dejando tal cual los positivos.

$$f(x) = \max(0, x) \begin{cases} 0 & \text{si } x < 0 \\ x & \text{si } x \geq 0 \end{cases} \quad (1.3)$$

Esta función, al anular los valores negativos, puede darse el fenómeno de neuronas muertas, lo que quiere decir que estas neuronas aportan el valor 0 independientemente del valor de entrada que se introduzca.

Esta función presenta buen rendimiento en redes convolucionales con imágenes. Además esta función no está acotada.

4. Función Leaky ReLU: esta función transforma los valores entrantes multiplicando los negativos por un coeficiente rectificativo y manteniendo tal cual los valores positivos.

$$f(x) = \begin{cases} \alpha x & \text{si } x \leq 0 \\ x & \text{si } x > 0 \end{cases} \quad (1.4)$$

Esta función es similar a la función ReLU, salvo porque esta función penaliza los valores negativos con un coeficiente que los rectifica.

5. Función Softmax: esta función transforma las salidas a una representación de probabilidades. Así pues, todas las probabilidades de las salidas suman 1.

$$f(Z)_j = \frac{e^{Z_j}}{\sum_{k=1}^K e^{Z_k}} \quad (1.5)$$

Esta función es útil para obtener una representación en forma de probabilidades, estando acotada entre 0 y 1. Además también es útil para normalizar datos de tipo multiclase y tiene un alto rendimiento en la capa de salida.

Para el etiquetado multiclase de partículas, la red neuronal más conveniente es la red neuronal *fully connected* o densamente conectada. Esta red se caracteriza, como se puede ver en la Figura 1, por tener cada neurona de una capa conectada a todas las demás neuronas de las capas anterior y posterior, siendo esta distribución la más sencilla [6].

A pesar de que este método de etiquetado por machine learning puede llegar a ser un tanto complejo debido a la posible radiación que generan las partículas de los jets que "manchan" las características de estas partículas (haciendo más difícil su clasificación), resulta más eficiente que los métodos tradicionales de clasificación, pues esta clasificación se basa en características hechas a mano propiciadas por la cromodinámica cuántica (teoría que rige la evolución de las partículas).

2 Objetivos

El objetivo principal de este trabajo es la implementación y optimización de una red neuronal *fully connected* en Python con la capacidad de clasificación de un gran número de jets de partículas en diez categorías de chorros, atendiendo a las propiedades de los jets y de las partículas que lo forman.

Con el conjunto de datos a gran escala *JetClass*, se tomarán datos de jets para entrenar y validar la red construida, estudiando así la función de pérdida (el error entre la predicción y la realidad). Además, para evitar el sobre-entrenamiento y que la red aporte excelentes resultados por haberse aprendido los datos del entrenamiento, se tomará otro conjunto de jets para realizar el test y comprobar si la red funciona adecuadamente.

Tras implementar una red genérica, se irá variando el número de neuronas por capa, el número de capas y la función de activación con el objetivo de optimizar la red todo lo posible, teniendo en cuenta el tiempo que invierte esta red en realizar la clasificación.

Finalmente, los resultados de la precisión obtenidos del mejor modelo se comparará con los aportados por los creadores de este conjunto de datos [7].

3 Contribuciones

Las principales contribuciones de este trabajo fin de grado a la Física de Partículas y al desarrollo de la inteligencia artificial son:

1. La automatización y el aumento de la eficiencia al etiquetar chorros de partículas. Debido a que el etiquetado de jets se hacía manualmente con la cromodinámica cuántica, al hacerlo mediante una red neuronal se observa una notable mejora en la eficiencia del etiquetado y, además, un ahorro de tiempo y esfuerzo considerable.
2. La optimización de las redes neuronales en Física de Partículas. Debido a que parte de este trabajo se basa en la optimización de la red neuronal, se experimenta con el ajuste de hiperparámetros de red para encontrar la más adecuada para etiquetar los jets, así como con la implementación de técnicas más avanzadas para evitar el sobre-entrenamiento.
3. La mejora en la precisión y la consistencia al clasificar los jets mediante redes neuronales, ya que se eliminan los posibles errores humanos.

De esta manera, se presentan los resultados que se han obtenido con el fin de comparar diferentes modelos de redes neuronales para obtener el que proporcione mayor rendimiento.

4 Materiales y metodología

4.1 Materiales

Para este trabajo fin de grado se han utilizado los *datasets* (o conjunto de datos) a gran escala *JetClass* para el entrenamiento, test y validación de una red neuronal densamente conectada (DNN) e implementada en Python con el fin de clasificar chorros de partículas subatómicas.

Estos datos se encuentran almacenados en archivos con formato *.root*, el cual es un formato creado por el CERN con el fin de almacenar todos los datos y poder manejarlos de forma eficiente. Cada fichero tiene una estructura de árbol, en el que de él salen nuevos directorios, archivos y otros objetos, distribuidos en ramas [8]. Los archivos con los que se trabaja tienen un árbol del que parten ramas, cada una con diferentes datos; algunas contienen etiquetas (en las que se clasificarán los jets) y el resto son las características.

JetClass contiene, como ya se ha dicho en la introducción (1), 100 millones de jets de partículas para el entrenamiento, 20 millones para la evaluación y 5 millones para la validación. Sin embargo, debido a la limitación de la memoria del ordenador con el que se está trabajando, se tomará un menor número de datos, trabajando entonces con el 50 % de los datos del entrenamiento (el resto de conjuntos sí se toman enteros).

4.2 Metodología

4.2.1 Paso previo a la realización del código: Análisis del contenido de los archivos

.root

Antes de comenzar, es importante conocer el contenido de los archivos que se van a utilizar; las etiquetas (*labels*) y las características (*features*). Para conseguir un programa más eficiente, es conveniente separar en dos variables las características de las etiquetas, por lo que se realiza un pequeño código para conocer el tipo y el nombre de las ramas:

```
1 import numpy as np
2 #Para trabajar en Python con archivos .root
3 import uproot
4
5 # Conocer el nombre y tipo de las ramas del archivo .root
6 nombre_fichprueba="HToBB_000.root"
7 fichprueba=uproot.open(nombre_fichprueba)
8
9 arbol=fichprueba["tree"] #Se asigna un nombre al arbol que hay en el archivo
10
11 #Muestra el tipo y descripción de las ramas
12 arbol.show()
```

Al compilar y ejecutar este código, se obtienen 41 ramas, 10 de ellas contienen

las etiquetas, cuyos datos son valores numéricos reales o enteros (*float* o *integer*) y datos de tipo *bool* (verdadero o falso), mientras que el resto contienen las características, que pueden ser escalares (enteros o reales) o vectores.

Esta diferenciación es importante tenerla en cuenta a la hora de leer y almacenar los datos en las variables que tomará la red.

4.2.2 Implementación y entrenamiento del modelo

Para poder clasificar todos los jets en 10 clases, se necesita implementar una red neuronal densamente conectada que se encargue de ello. Para ello, se explicará paso a paso la realización de un código en Python que desempeñe esta función.

Importar librerías [9]

En primer lugar, se importa una serie de bibliotecas que permiten trabajar con los datos del conjunto *JetClass* y analizarlos:

- La librería Tensorflow [10], una biblioteca creada por Google, servirá para poder implementar el modelo de la red neuronal de interés (en este caso la DNN), y poder entrenarla.
- De `sklearn.preprocessing` se importará `StandardScaler` [11] con el objetivo de normalizar los datos para conseguir una mayor convergencia en el entrenamiento. Normalizar los datos consigue uniformar los gradientes y optimizar la convergencia.
- La librería `numpy` permite trabajar con una amplia cantidad de datos (booleanos, enteros, reales, matrices, etc.) y contiene además gran cantidad de operaciones matemáticas.
- La librería `uproot` será necesaria para poder trabajar con los archivos proporcionados, cuyo formato es `.root`.
- Debido a que se ha trabajado en un ordenador cuyo sistema operativo es iOS, a la hora de leer los ficheros el programa tomaba una serie de archivos que crea el propio sistema para poder mostrar los archivos en el escritorio (llamados `.DS_Store` [12]), por lo que se ha tenido que añadir la librería `os`. Esta biblioteca permite interactuar con el sistema operativo, manejar estos archivos y conseguir que el programa no los tenga en cuenta.

Inicialización de variables

Tras haber importado todas las bibliotecas necesarias para que funcione el código correctamente, se inicializan las variables que indican la dirección de los directorios donde se encuentran los archivos. Además, se inicializan las variables que contengan los archivos [13], [14]; por un lado los de entrenamiento, por otro los de prueba y por otro los de validación.

Finalmente, se leen los datos contenidos mediante la función `leer_archivo`. Esta función toma cada una de las características, en una variable las vectoriales

[15] y en otra las escalares, y las concatena para obtener una única variable con todas las características. Por otra parte, toma cada etiqueta y las concatena también en una sola variable.

Normalización de datos

Ahora, se van a escalar las características para conseguir ajustar los datos de tal modo que tengan una media de 0 y una desviación típica de 1, mejorando así el rendimiento de este modelo.

Creación y compilación de la red neuronal [16]

A continuación, se realiza la configuración del modelo, donde se establece la capa de entrada con los datos de entrenamiento. Posteriormente, se añaden las capas ocultas, con un número de neuronas que pueden ir variando y con una función de activación que también se irá alterando para estudiar su optimización. Finalmente, se establece la capa de salida, con un número fijo de neuronas (en este caso 10 debido a que son las categorías en las que se clasifican los jets de partículas) y con la función de activación Softmax (1.5), una función que se utiliza en la capa de salida para la clasificación multiclase, convirtiendo los valores de salida de la capa anterior en probabilidades que suman 1 [17].

Además, para evitar el sobre-entrenamiento, se ha añadido entre capa y capa la función Dropout, que se encarga, bajo una probabilidad (en este caso del 50 %), de eliminar neuronas para que la red no aprenda tan específicamente los datos que se le aportan [18].

Compilación del modelo

Una vez establecido el modelo, se compila, estableciendo el optimizador adam [19], mejorando el proceso de aprendizaje de la red.

Además, se añade la función de pérdida CategoricalCossentropy [20], con el objetivo de poder calcular el error que se comete entre la predicción que hace el modelo y la realidad. En concreto, se aplica esta función debido a la naturaleza de los datos, ya que trabaja con datos de distinta naturaleza.

Por último, se añade una métrica para que calcule la precisión del modelo.

Entrenamiento de la red

Una vez realizado todo lo anterior, queda entrenar el modelo. Para ello, se implementa el comando `fit`, tomando los datos y etiquetas del entrenamiento y de validación y se establece el número de épocas que permanecerán fijas durante todo el estudio. Una época se define como la interacción completa por todos los datos, es decir, cuando se han procesado todos los datos una vez [21].

Cálculo del error de predicción

Para finalizar con la parte del entrenamiento, se anota el error entre la predicción y la realidad y se calcula la desviación típica muestral. Estos valores servirán para comprobar la eficiencia de red, estudiando cómo se puede optimizar este modelo según el valor de esta función de pérdida.

A continuación, se refleja el código que se ha obtenido tras seguir todos los pasos descritos previamente:

```

1  import numpy as np
2  #Para manejar los archivos .root:
3  import uproot
4  #Para trabajar en sistema operativo iOS:
5  import os
6  #Para construir la red neuronal:
7  import tensorflow as tf
8  from tensorflow.keras import layers, models
9  #Para normalizar los datos
10 from sklearn.preprocessing import StandardScaler
11
12 # Definir rutas de archivos de entrenamiento, prueba y validación
13 dir_entreno = "/train_100M_part0"
14 dir_val= "/val_5M"
15 dir_test = "/test_20M"
16
17 def leer_archivo(camino):
18     archivo= uproot.open(camino)
19     arb=archivo["tree"]
20
21     #Leer características escalares
22     cesc = [
23         #Características de jets
24         arb["jet_pt"], arb["jet_eta"],
25         arb["jet_phi"], arb["jet_energy"],
26         arb["jet_nparticles"], arb["jet_sdmass"],
27         arb["jet_tau1"], arb["jet_tau2"],
28         arb["jet_tau3"], arb["jet_tau4"],
29
30         #Características auxiliares
31         arb["aux_genpart_eta"], arb["aux_genpart_phi"],
32         arb["aux_genpart_pid"], arb["aux_genpart_pt"],
33         arb["aux_truth_match"]
34     ]
35
36     # Leer las características vectoriales
37     cvec = [
38         #Características de cada partícula
39         #Componentes x, y, z
40         np.array([np.sum(i) for i in
41         arb["part_px"].array(library="np")]),
42         np.array([np.sum(i) for i in
43         arb["part_py"].array(library="np")]),
44         np.array([np.sum(i) for i in
45         arb["part_pz"].array(library="np")]),
46         #Energía, deta, dphi
47         np.array([np.sum(i) for i in
48         arb["part_energy"].array(library="np")]),
49         np.array([np.sum(i) for i in
50         arb["part_deta"].array(library="np")]),

```

```

51     np.array([np.sum(i) for i in
52     arb["part_dphi"].array(library="np")]),
53     #Valor y error de d0, dz
54     np.array([np.sum(i) for i in
55     arb["part_d0val"].array(library="np")]),
56     np.array([np.sum(i) for i in
57     arb["part_d0err"].array(library="np")]),
58     np.array([np.sum(i) for i in
59     arb["part_dzval"].array(library="np")]),
60     np.array([np.sum(i) for i in
61     arb["part_dzerr"].array(library="np")]),
62     #Carga, hadrón cargado o no
63     np.array([np.sum(i) for i in
64     arb["part_charge"].array(library="np")]),
65     np.array([np.sum(i) for i in
66     arb["part_isChargedHadron"].array(library="np")]),
67     np.array([np.sum(i) for i in
68     arb["part_isNeutralHadron"].array(library="np")]),
69     #¿Es fotón, electrón, muón?
70     np.array([np.sum(i) for i in
71     arb["part_isPhoton"].array(library="np")]),
72     np.array([np.sum(i) for i in
73     arb["part_isElectron"].array(library="np")]),
74     np.array([np.sum(i) for i in
75     arb["part_isMuon"].array(library="np")])
76 ]
77
78 # Concatenar características
79 caract = np.column_stack(cesc + cvec)
80
81 # Leer y almacenar etiquetas de clasificación
82 etiq = np.stack([
83     arb["label_QCD"], arb["label_Hbb"],
84     arb["label_Hcc"], arb["label_Hgg"],
85     arb["label_H4q"], arb["label_Hqq1"],
86     arb["label_Zqq"], arb["label_Wqq"],
87     arb["label_Tbqq"], arb["label_Tbl"]
88 ], axis=-1)
89
90     return caract, etiq
91
92
93
94 # Obtener la lista de archivos en cada directorio, excluyendo archivos .DS_Store
95 arch_entreno= [os.path.join(dir_entreno, f) for f in os.listdir(dir_entreno)
96                 if f.endswith('.root') and not f.startswith('.')]
97 arch_val = [os.path.join(dir_val, f) for f in os.listdir(dir_val)
98             if f.endswith('.root') and not f.startswith('.')]
99 arch_test=[os.path.join(dir_test, f) for f in os.listdir(dir_test)
100            if f.endswith('.root') and not f.startswith('.')]
101
102
103 # Leer los datos de entrenamiento, validación y evaluación

```

```
104 datos_entreno=[]
105 etiq_entreno=[]
106 for camino in arch_entreno:
107     caract, etiq = leer_archivo(camino)
108     datos_entreno.append(caract)
109     etiq_entreno.append(etiq)
110
111 datos_entreno = np.vstack(datos_entreno)
112 etiq_entreno = np.vstack(etiq_entreno)
113
114 datos_val=[]
115 etiq_val = []
116 for camino in arch_val:
117     caract, etiq = leer_archivo(camino)
118     datos_val.append(caract)
119     etiq_val.append(etiq)
120
121 datos_val = np.vstack(datos_val)
122 etiq_val = np.vstack(etiq_val)
123
124 # Leer los datos de prueba
125 datos_test=[]
126 etiq_test=[]
127 for camino in arch_test:
128     caract, etiq = leer_archivo(camino)
129     datos_test.append(caract)
130     etiq_test.append(etiq)
131
132 datos_test = np.vstack(datos_test)
133 etiq_test = np.vstack(etiq_test)
134
135 escalar = StandardScaler()
136 #Escalar los datos de entrenamiento
137 datos_entreno = escalar.fit_transform(datos_entreno)
138 #De validación
139 datos_val = escalar.transform(datos_val)
140 #Y de test
141 datos_test = escalar.transform(datos_test)
142
143 #Implementación del modelo de la red
144 #Número de neuronas por capa oculta
145 N=10
146 # Definir modelo DNN
147 modelo = models.Sequential([
148     #Capa de entrada
149     layers.Input(shape=(datos_entreno.shape[1],)),
150     #Capas ocultas
151     layers.Dense(N, activation="relu"),
152     #Capa para apagar neuronas y evitar sobre-ajuste
153     layers.Dropout(0.5),
154     layers.Dense(N,activation="relu"),
155     layers.Dropout(0.5),
156     # Capa de salida por ser 10 tipos son 10 neuronas
```

```

157     layers.Dense(10, activation="softmax")
158 ])
159
160 # Compilar calculando la función pérdida y la precisión
161 modelo.compile(optimizer="adam",
162                #Se usa esta función para este tipo de datos
163                loss="categorical_crossentropy",
164                #Para calcular la precisión
165                metrics=["accuracy"])
166 #Entrenar el modelo
167 historial=modelo.fit(datos_entreno, etiq_entreno, epochs=5,
168                     datos_val=(datos_val, etia_val))
169
170 #Calcula la función de pérdida y la refleja en pantalla
171 perdida=historial.history["loss"][-1]
172 print(f"Función pérdida: {perdida}")
173
174 #Cálculo de la desviación típica
175 #Primero se toma la predicción
176 ypredentreno = modelo.predict(datos_entreno)
177 #Luego el error entre predicción y realidad
178 errentreno = ypredentreno.flatten() - etiq_entreno.flatten()
179 #Se calcula la desviación con la función de Python
180 desventreno = np.std(errentreno, ddof=1) # ddof=1 para desviación muestral
181 print(f"Desviación típica: {desventreno}")

```

4.2.3 Evaluación del modelo

Una vez obtenido y entrenado el modelo de red neuronal más eficiente y optimizado posible se ha de evaluar, con el fin de comprobar que este modelo realiza correctamente su cometido y no se encuentra sobre-entrenado.

Como se ha mostrado en el código anterior, ya se han leído los archivos y escalado los datos del directorio donde se encuentran los datos de test. Es importante no enseñarle a la red estos datos hasta que se vaya a evaluar, con la intención de que no se aprenda estos datos.

De esta forma, se añaden las siguientes líneas de código para evaluar el modelo:

```

1 #Finalmente, evaluación del modelo
2 tloss, taccuracy = modelo.evaluate(datos_test, etiq_test) #Función de Python
3 print(f"Pérdida: {tloss}")
4 print(f"Precisión: {taccuracy}")
5 #Con el conjunto de test repite el proceso para la desviación tomando la predicción:
6 ypredtest=modelo.predict(datos_test)
7
8 #La diferencia entre predicción y realidad
9 errtest = ypredtest.flatten() - etiq_test.flatten()
10
11 #Y la desviación típica final
12 desvtest = np.std(errtest, ddof=1)
13 #Para la desviación típica muestral se tomaba ddof=1

```

```
14 print(f"Desviación típica (test): {desvtest}")
```

5 Resultados y discusión

La eficiencia de un modelo de red neuronal depende de numerosos factores, como son el número de neuronas, el tipo de función de activación, el número de capas ocultas, etc. Por ello, para obtener un modelo lo más eficiente posible, se van a ir modificando algunas variables mientras otras se van a mantener fijas, estudiando el valor de la función de pérdida de entrenamiento que calcula la red. Además, es importante tener en cuenta el tiempo que invierte en entrenar la red, ya que no es eficiente un modelo que, aunque dé buenos resultados, tarde demasiado en ejecutarse.

Para ello, se van a realizar varias pruebas en las que se mantienen fijas todas las variables salvo una. De esta manera, se le otorga un valor a esta variable, se compilará y ejecutará el modelo y se entrenará 6 veces, anotando el valor final del error calculado entre la predicción y el valor real (la pérdida), su desviación típica y el tiempo invertido en cada entrenamiento. Posteriormente, se cambia el valor de la variable y se repite el proceso.

Con las 6 repeticiones se calcula el valor medio de la pérdida (con su correspondiente desviación típica) y el tiempo medio que tarda el modelo en entrenar, pudiendo así comparar qué modelo ofrece mejores resultados.

5.1 Entrenamiento del modelo

5.1.1 Variación del número de neuronas

En primer lugar, se comienza variando el número de neuronas de 10 a 60 aumentando de 10 en 10. Mientras tanto, se mantiene fijo el número de capas ocultas en 3 capas (excluyendo la capa de salida) y el tipo de función de activación, en este caso, función ReLU. Además, se mantiene constante durante todas las simulaciones la capa de salida, su función de activación (Softmax) y el número de neuronas a 10 (cada una para cada clase).

Con todo lo descrito anteriormente, se han obtenido los resultados mostrados en la Tabla 1:

# de neuronas	Pérdida	Desviación típica	Tiempo (minutos)
10	1,19	0,186	42,5
20	0,433	0,104	42,82
30	0,152	0,0160	44,0
40	0,104	0,0102	44,3
50	0,103	0,0164	49,4
60	0,102	0,0148	50,9

Tabla 1: Valores del error entre la predicción hecha por la red neuronal (con 3 capas ocultas y con función de activación ReLU) y el dato real del archivo, con su desviación típica y del tiempo invertido para cada tipo de modelo.

Como se puede observar en esta Tabla, hay varios factores determinantes para elegir el número de neuronas que más optimice el modelo.

Por un lado, se observa claramente cómo la función de pérdida disminuye considerablemente hasta que, a partir de 40 neuronas se estanca, disminuyendo únicamente un 0,97 % entre las 40 y las 50 neuronas y de las 50 a las 60 neuronas. Además, para las desviaciones típicas para 50 y 60 neuronas se observa un aumento de su valor con respecto al que se obtiene con 40 neuronas (un aumento del 37 % y 31 % respectivamente). Esto puede deberse a que se ha producido un sobre-ajuste, es decir, la red se ha especializado en los datos de entrenamiento y no sabe generalizar, produciendo este aumento en la desviación típica. Por otro lado, al pasar de las 40 neuronas también se observa un aumento en el tiempo que invierte la red en realizar el entrenamiento.

Con estas condiciones, se puede considerar que el número de neuronas que más optimiza el modelo es de 40 neuronas, donde la función de pérdida es la más baja con la más baja desviación típica y el tiempo invertido no varía demasiado con respecto a las 30 neuronas (un 0,68 %).

5.1.2 Variación del número de capas ocultas

Una vez determinado el número de neuronas para las cuales se obtienen mejores resultados (en este caso 40), se cambia el número de capas ocultas, pasando de 3 a 7 y añadiendo una en cada experimento. De este modo, se han reflejado en la Tabla 2 los resultados obtenidos tras hacer estos cambios, manteniendo constante el número de neuronas y el tipo de función de activación.

# de capas	Pérdida	Desviación típica	Tiempo (minutos)
3	0,104	0,0102	44,3
4	0,220	0,056	48,4
5	0,448	0,174	52,5
6	0,468	0,187	55,5
7	0,618	0,242	57,3

Tabla 2: Resultados de la pérdida, su desviación típica y del tiempo invertido para cada experimento, variando el número de capas y manteniendo constante el número de neuronas en 40 y la función de activación ReLU.

En esta Tabla se puede comprobar claramente cómo aumenta la función de pérdida conforme se añaden capas neuronales, duplicándose aproximadamente el valor de la pérdida entre el modelo con 3 capas ocultas y 4 capas ocultas. También cabe destacar que la desviación típica y el tiempo invertido también aumentan conforme lo hacen el número de capas neuronales ocultas.

5.1.3 Variación del tipo de función de activación

Por último, queda variar el tipo de función de activación, mientras se mantiene constante el número de neuronas a 40 y el número de capas ocultas a 3. De esta manera, se repite el mismo proceso para las funciones Sigmoide, tangente hiperbólica (o Tanh), ReLU, Leaky ReLU y Softmax, obteniendo los valores que se representan en la Tabla 3:

Función de activación	Pérdida	Desviación típica	Tiempo (minutos)
Sigmoide	0,130	0,0138	52,5
Tanh	0,145	0,00590	48,3
ReLU	0,104	0,0102	44,3
Leaky ReLU	0,117	0,00583	48,9
Softmax	0,105	0,210	51,6

Tabla 3: Valores medios obtenidos de la pérdida, la desviación típica y el tiempo invertido variando el tipo de función de activación, mientras se mantiene constante el número de neuronas (40 neuronas) y el número de capas ocultas (3 capas).

Las funciones que peor resultado arrojan, como se puede ver, son las funciones Sigmoide, Tanh y Leaky ReLU.

La función Sigmoide provoca que los pesos sinápticos, que se van actualizando conforme la red va entrenando, lo hagan de manera muy lenta, provocando que las neuronas de las capas ocultas aprendan más lentamente que en los casos en los que hay otras funciones de activación.

Para la función Tanh ocurre lo mismo que con la función anterior. Además, como el rango para esta función es de $(-1,1)$, puede ocurrir que estos extremos se saturen y también se ralentice el decaimiento de la función de pérdida, por lo que tiene sentido que se haya obtenido un valor de la pérdida mayor para esta función que

para la función Sigmoide.

Por último, quedan las funciones ReLU y Softmax, las cuales arrojan resultados realmente parecidos. Sin embargo, la función Softmax en este tipo de redes (cuya finalidad es clasificar distintas categorías) carece de sentido utilizarla en las capas ocultas, ya que esta función está creada para la capa de salida, pudiendo generar un deterioro en el rendimiento del modelo. Además, teniendo en cuenta la desviación típica, se obtiene un valor considerablemente mayor que el valor de la propia función de pérdida, descartando completamente la validez de este resultado. Por tanto, se elige la función ReLU como la función de activación en las capas ocultas.

5.2 Evaluación del modelo final

El modelo definitivo más optimizado resulta tener la siguiente estructura:

```

1  #Modelo definitivo de red
2  #Número de neuronas
3  N=40
4
5  # Red DNN
6  modelo = models.Sequential([
7      #Capa de entrada
8      layers.Input(shape=(datos_entreno.shape[1],)),
9      #Capas ocultas
10     layers.Dense(N, activation="relu"),
11     #Capa para evitar el sobre-ajuste
12     layers.Dropout(0.5),
13     layers.Dense(N, activation="relu"),
14     layers.Dropout(0.5),
15     layers.Dense(N, activation="relu"),
16     layers.Dropout(0.5),
17     #Capa de salida
18     layers.Dense(10, activation="softmax")
19 ])
20

```

Sin embargo, para saber si realmente este modelo es eficiente es necesario evaluarlo con el conjunto de datos que se ofrece para su test. Con esta evaluación se puede estudiar si la red construida sabe generalizar con el entrenamiento que ha recibido (ha entrenado 6 veces), o si, por el contrario, ha ido aprendiendo los patrones de los datos del entrenamiento y se encuentra sobre-entrenado.

Así pues, añadiendo las líneas de código que se muestran en la metodología en el apartado (4.2.3), se le enseña al modelo por primera vez los datos (es importante que sea la primera vez que el modelo trabaja con estos datos para que no tenga la oportunidad de aprenderlos) y se estudia el valor que se obtiene para la función de pérdida y su desviación típica. En la Tabla 4 se observan estos resultados junto con los obtenidos en el entrenamiento para que sea más claras su comparación.

	Función pérdida	Desviación típica
Entrenamiento	0,104	0,0102
Evaluación	0,151	0,0162

Tabla 4: Valores de la función de pérdida y su desviación típica para el entrenamiento y la evaluación de un modelo con 40 neuronas , 3 capas ocultas y función de activación ReLU (más la capa de salida con función de activación Softmax).

Como se puede observar, el valor de la función de pérdida de la evaluación sigue siendo cercano a 0, por lo que la red es eficiente y sabe trabajar bien con los datos. Sin embargo, sí es notable un aumento entre este valor y el que se obtiene en el entrenamiento, por tanto, se puede deducir que, a pesar de haber añadido al modelo las capas con Dropout para evitarlo, la red se encontraba un tanto sobreajustada, aunque se han obtenido buenos resultados.

5.3 Comparación con los resultados aportados por los creadores de *JetClass*

Además de aportar la función de pérdida y la desviación típica muestral de la red neuronal, este código también aporta la precisión con la que el modelo predice la clasificación. De esta manera, se puede comparar el resultado de la predicción de la evaluación con la precisión que obtienen los creadores del conjunto de datos *JetClass* en su estudio [7]. De esta forma, se muestran en la Tabla 5 los valores de la precisión para los modelos realizados por Reza Abbasi, Aditya Anand, Matteo Faccioli, Raghav Kansal, Gregor Kasieczka y Shih-Chieh Hsu (estos son ParticleNet y ParT) en comparación con el modelo realizado en este trabajo.

Modelo	Precisión
PartibleNet	85,6 %
ParT	88,9 %
Red neuronal	94,4 %

Tabla 5: Valores de la precisión para los modelos ParticleNet, ParT y el modelo de red neuronal de este trabajo.

Hay que tener en cuenta que el modelo construido no ha sido entrenado con todos los datos del conjunto debido a la falta de memoria del ordenador, por lo que esto afecta a los resultados en la precisión. Teóricamente, una menor cantidad de datos provocaría una menor precisión, puesto que así le sería más complicado a la red poder generalizar [22]. Sin embargo, al evaluar la red se observa una precisión mayor que para los otros modelos, lo que puede deberse a diferentes causas [23]. Es posible que este modelo sea más eficiente tomando las características de los jets a pesar de tener una menor cantidad de datos. Además, al añadir la función Dropout permite una mejor regularización y, por tanto, un mayor rendimiento.

6 Conclusiones

Tras realizar este trabajo, se puede concluir que, para clasificar un conjunto de datos sobre jets de partículas subatómicas con diversas características en varias categorías según la procedencia de los jets (en concreto, 10 clases), lo más conveniente es construir un modelo de red neuronal densamente conectada con diversas propiedades.

En primer lugar, se determina una red con una capa de salida con 10 neuronas, debido a que los jets se clasifican en 10 tipos de categorías. Por otra parte, la función de activación elegida para esta capa es Softmax, ya que esta función se ha creado específicamente para esta capa por su alto rendimiento y por su capacidad de normalizar los datos multiclase.

En segundo lugar, para las capas ocultas se ha concluido que la mejor combinación, por su alto rendimiento, eficiencia y por el bajo tiempo que invierte el modelo en entrenar comparado con otras combinaciones, es la reflejada en el apartado (5.2), la que consta de tres capas ocultas, 40 neuronas en cada capa y la función de activación ReLU.

Finalmente, y a pesar de que se ha observado un pequeño sobre-entrenamiento al evaluar el modelo (su función de pérdida es mayor que en el entrenamiento), ha sido importante también añadir entre capa oculta y capa oculta otra capa con la función Dropout, puesto que, al ir apagando neuronas con el 50 % de probabilidad, se reduce el sobre-ajuste, aumentando la eficiencia y el rendimiento de la red. Así pues, se ha construido un modelo más eficiente que los propuestos por los creadores del conjunto de datos *JetClass*, a pesar de haber utilizado menos datos para entrenar el modelo.

7 Agradecimientos

La realización de este trabajo no hubiera sido posible sin la inestimable ayuda de mis tutores, José Luis Bernier y Luis Javier Herrera, por haberme orientado y guiado durante la realización de este proyecto, convirtiéndolo en una experiencia enriquecedora. Agradecimientos también a mi familia, por todo el apoyo y afecto que he recibido durante todo este tiempo. Por último, agradecerles a mis amigos y compañeros de clase ese gran soporte recibido día a día.

Referencias

- [1] J. Doe, "Quantum chromodynamics (qcd)," 2024. <https://www.sciencedirect.com/topics/physics-and-astronomy/quantum-chromodynamics>.
- [2] Qu, "Jetclass: A large-scale dataset for deep learning in jet physics." <https://paperswithcode.com/dataset/jetclass>.
- [3] E. Acevedo, A. Serna, and E. Serna, "Principios y características de las redes neuronales artificiales," *Desarrollo e innovación en ingeniería*, vol. 173, 2017.
- [4] Moreno Alberola, "Reconocimiento de objetos en un robot social," 2018. https://rua.ua.es/dspace/bitstream/10045/80410/1/Reconocimiento_de_objetos_en_un_robot_social_MORENO_ALBEROLA_ALVARO.pdf.
- [5] D. Calvo, "Función de activación – redes neuronales," 2018. <https://www.diegocalvo.es/funcion-de-activacion-redes-neuronales/>.
- [6] In2AI, "Todo lo que debes saber sobre las redes neuronales," 2024. [https://www.google.es/url?sa=t&source=web&rct=j&opi=89978449&url=https://in2ai.com/todo-lo-que-debes-saber-sobre-las-redes-neuronales/%23%3A~:text=3DRedes%2520Neuronales%2520Densamente%2520Conectadas%2520\(Fully,posterior%2520C%2520siendo%2520las%2520m%2520C%2520A1s%2520sencillas.&ved=2ahUKEwjQiZC91KuGAXWfVKQEHZW0ANoQFnoECBEQAw&usg=A0vVaw2xoMM4GWsU2joSHd-G-2Ms](https://www.google.es/url?sa=t&source=web&rct=j&opi=89978449&url=https://in2ai.com/todo-lo-que-debes-saber-sobre-las-redes-neuronales/%23%3A~:text=3DRedes%2520Neuronales%2520Densamente%2520Conectadas%2520(Fully,posterior%2520C%2520siendo%2520las%2520m%2520C%2520A1s%2520sencillas.&ved=2ahUKEwjQiZC91KuGAXWfVKQEHZW0ANoQFnoECBEQAw&usg=A0vVaw2xoMM4GWsU2joSHd-G-2Ms).
- [7] Qu, "Jetclass: A large-scale dataset for deep learning in jet physics," *arXiv preprint arXiv:2202.03772*, 2022. <https://arxiv.org/pdf/2202.03772>.
- [8] Andrés, "Root's basics ii: ¿qué son los archivos .root y cómo crear uno?," 2016. https://www.google.es/url?sa=t&source=web&rct=j&opi=89978449&url=https://plaintextfiles.wordpress.com/2016/04/16/roots-basics-ii-que-son-los-archivos-root-y-como-crear-uno/&ved=2ahUKEwi-vcP5g6uGAXVfRqQEhQAMdAMQFnoECBUQAQ&usg=A0vVaw3N_5c9s4mLaHUIJafS_5BkT.
- [9] I. Challenger-Pérez and Díaz-Ricardo, "El lenguaje de programación python," *Ciencias Holguín*, vol. 20, no. 2, pp. 1–13, 2014.
- [10] J. Torres, *Python deep learning: Introducción práctica con Keras y TensorFlow 2*. Alpha Editorial, 2020.
- [11] scikit-learn developers, "Preprocessing data," 2023. <https://scikit-learn.org/stable/modules/preprocessing.html>.
- [12] M. Support, "Quitar .ds_store archivos en macos," 2023. <https://support.microsoft.com/es-es/office/quitar-ds-store-archivos-en-macos-d2f8dca0-740a-4f7e-89b9-5b2cbbc50386>.

- [13] GeeksforGeeks, "Python os.path.join() method," 2023. <https://www.geeksforgeeks.org/python-os-path-join-method/>.
- [14] T. N. Community, "numpy.column_stack," 2023. https://numpy.org/doc/stable/reference/generated/numpy.column_stack.html.
- [15] U. de Stack Overflow, "Sumar enteros de un array en python," 2017. <https://es.stackoverflow.com/questions/167201/sumar-enteros-de-un-array-en-python>.
- [16] Na8, "Una sencilla red neuronal en python con keras y tensorflow," 2018. https://www.google.es/url?sa=t&source=web&rct=j&opi=89978449&url=https://www.aprendemachinelearning.com/una-sencilla-red-neuronal-en-python-con-keras-y-tensorflow/&ved=2ahUKEwi--bjI7a0GAxXC_gIHHem5B78QFnoECBcQAQ&usg=A0vVaw3ZZ7bBdKCAuo9Gp3slrGe.
- [17] javi, "Función softmax: Activación para la clasificación," 2023. <https://www.google.es/url?sa=t&source=web&rct=j&opi=89978449&url=https://jacar.es/funcion-softmax-activacion-para-la-clasificacion/%23:~:text=%3DLa%2520funci%25C3%25B3n%2520SoftMax%2520es%2520una%2520funci%25C3%25B3n%2520de%2520activaci%25C3%25B3n%2520que%2520se,en%2520probabilidades%2520que%2520suman%2520uno.&ved=2ahUKEwjewpzkuyGAXWuaqQEHXc7Cb4QFnoECBQQA&usg=A0vVaw14BaQd54ypaEHh91p7xomA>.
- [18] A. Antona Castañares, "Aplicación del dropout a la cuantificación de la incertidumbre en redes neuronales," 2020. <https://www.google.es/url?sa=t&source=web&rct=j&opi=89978449&url=https://oa.upm.es/57875/%23:~:text=%3DEl%2520dropout%2520es%2520una%2520t%25C3%25A9cnica,de%2520regularizaci%25C3%25B3n%2520de%2520los%2520modelos.&ved=2ahUKEwj92Y-Bq62GAxUE8QIHHAeoC9cQFnoECBEQA&usg=A0vVaw399LCJoDnvSTWJ09EPx0JN>.
- [19] Interactive Chaos, "Tutorial de machine learning: Adam," 2023. <https://interactivechaos.com/es/manual/tutorial-de-machine-learning/adam>.
- [20] I. Gavilán, "Catálogo de componentes de redes," 2020. http://bluechip.ignaciogavilan.com/2020/05/catalogo-de-componentes-de-redes_25.html.
- [21] V. Morales, "Redes neuronales," 2023. https://bookdown.org/victor_morales/TecnicasML/redes-neuronales.html.
- [22] Contenidos.ai, "¿qué son los datos de entrenamiento o training data en ia?," n.d. [https://www.google.es/url?sa=t&source=web&rct=j&opi=89978449&url=https://www.contents.ai/es/magazine/inteligencia-artificial/que-son-los-datos-de-entrenamiento-o-training-data-en-ia/%23:~:text=%3DLos%2520datos%2520de%2520entrenamiento%2520son%2520de%2520crucial%2520importancia%2520para%](https://www.google.es/url?sa=t&source=web&rct=j&opi=89978449&url=https://www.contents.ai/es/magazine/inteligencia-artificial/que-son-los-datos-de-entrenamiento-o-training-data-en-ia/%23:~:text=%3DLos%2520datos%2520de%2520entrenamiento%2520son%2520de%2520crucial%2520importancia%2520para%2520)

2520el,puede%2520llegar%2520a%2520ninguna%2520parte.
&ved=2ahUKEwiwoqvayY-HAxUFUaQEhbW2CZ8QFnoECA8QAw&usg=
A0vVaw0CrOp-jXoNA9-GXzrimJ2-.

- [23] A. W. Services, "Improving model accuracy," n.d.
https://www.google.es/url?sa=t&source=web&rct=j&opi=89978449&url=https://docs.aws.amazon.com/es_es/machine-learning/latest/dg/improving-model-accuracy.html&ved=2ahUKEwi2mpGS0o-HAxU6zwIHHTzHBncQFnoECB4QAAQ&usg=A0vVaw3pkomLnMA1Ums6_EuuBwfe.



1. DATOS BÁSICOS DEL TFG:

Título: Detección de obstáculos en tiempo real mediante técnicas de Machine Learning

Descripción general (resumen y metodología):

Estudio e implementación de metodologías para la detección de obstáculos en tiempo real. El motivo es el de desarrollar un dispositivo de ayuda para las personas con discapacidad visual.

Metodología:

- Estudiar la problemática y desarrollar un modelo
- Analizar distintos frameworks y herramientas para implementación de algoritmos de Machine Learning
- Estudiar técnicas de Machine Learning basadas en aprendizaje
- Implementación de algoritmos en un lenguaje de programación adecuado (Python)
- Analizar los resultados

Tipología: Trabajos experimentales, de toma de datos de campo o de laboratorio.

Objetivos planteados:

- Estudio de alternativas para la detección de obstáculos
- Implementación de metodologías basadas en Machine Learning
- Comparación de resultados
- Optimización para uso en tiempo real
- Diseño de un prototipo de dispositivo

Bibliografía básica:

- Redes Neuronales & Deep Learning (Spanish Edition) 2018 Spanish Edition. Fernando Berzal.
- Deep Learning. Ian Goodfellow and Yoshua Bengio and Aaron Courville. MIT Press, 2016.
- Obstacle Detection Display for Visually Impaired: Coding of Direction, Distance, and Height on a Vibrotactile Waist Band. 2017. Jan B. F. van Erp, Liselotte C. M. Kroon, Tina Mioch and Katja I. Paul. Frontiers in ICT. <https://doi.org/10.3389/fict.2017.00023>

Recomendaciones y orientaciones para el estudiante:

Plazas: 1

2. DATOS DEL TUTOR/A:

Nombre y apellidos: JOSÉ LUIS BERNIER VILLAMOR

Ámbito de conocimiento/Departamento: ARQUITECTURA Y TECNOLOGÍA DE COMPUTADORES

Correo electrónico: jbernier@ugr.es

3. COTUTOR/A DE LA UGR (en su caso):

Nombre y apellidos: LUIS JAVIER HERRERA MALDONADO

Ámbito de conocimiento/Departamento: ARQUITECTURA Y TECNOLOGÍA DE COMPUTADORES

Correo electrónico: jherrera@ugr.es

4. COTUTOR/A EXTERNO/A (en su caso):

Nombre y apellidos:

Correo electrónico:

Nombre de la empresa o institución:

Dirección postal:

Puesto del tutor en la empresa o institución:

5. DATOS DEL ESTUDIANTE:

Nombre y apellidos: ANA FUENTES RODRIGUEZ

Correo electrónico: afuentesr2019@correo.ugr.es